

Quantum Minimum Searching Algorithms for Active User Detection in Wireless IoT Networks — 해설·정리본

항목	내용
원문 제목	Quantum Minimum Searching Algorithms for Active User Detection in Wireless IoT Networks
저자	Muhammad Idham Habibie, Claire Goursaud, Jihad Hamie (CITI Laboratory EA3720, Univ Lyon,
게재지 / 연도	IEEE Internet of Things Journal, Vol. 11, No. 12, pp. 22603-22615, 2024년 6월 15일
DOI	10.1109/JIOT.2024.3382337
분량	13쪽 (본문 12쪽 + 참고문헌·저자 약력 1쪽), 2단 조판
원문 파일	../papers/4. Grover 응용 논문/Quantum_Minimum_Searching_Algorithms_for_Active_User

페이지 표기 안내: 이 문서의 원문 p.N 은 PDF 뷰어의 쪽 번호(1~13)입니다. 논문 지면 번호와는
지면 = 22602 + N 관계입니다. 예를 들어 PDF p.5 = 지면 22607.

한눈에 보기

- 5G / IoT 기지국은 단말이 접속 허가(grant) 없이 그냥 쏘아 올린 신호 더미를 받아, “지금 누가 켜져 있나”를 실시간으로 알아내야 합니다. 이것이 **활성 사용자 검출(active user detection, AUD)** 이고, 최적 검출기인 ML은 2^N 가지 활성 조합을 전부 뒤져야 해서 $O(2^N)$ 입니다.
- 이 논문은 ML을 “정렬 안 된 목록에서 최솟값 찾기” 로 다시 쓰고, 그로버 기반 최솟값 탐색 알고리즘인 **Dürr-Høyer(DHA)** 로 $O(\sqrt{2^N})$ 에 풉니다. 오라클(oracle)이 “인덱스가 정답이냐”가 아니라 “거리 $\|y - b \cdot C\|_2^2$ 가 임계값 δ 보다 작으냐” 를 판정하는 비교기(comparator)라는 점이 핵심입니다.
- 기여는 두 가지입니다. (1) **IIMSA** — 해의 개수 S 를 시스템 통계로 미리 추정해, 그로버 반복 횟수를 DHA처럼 무작위로 뽑지 않고 최적값 L_{opt} 로 정확히 잡습니다. (2) **CCR/ZF 하이브리드** — 고전 수신기로 임계값 δ 를 좋은 값에서 출발시킵니다.
- 결과적으로 **ZF-IIMSA** 가 성공확률(success probability)을 유지하거나 약간 올리면서 그로버 반복 횟수를 크게 줄입니다. SF(사용자 수)가 클수록 이득이 커지고, 잡음 있는 양자 하드웨어에서도 우위가 유지됩니다.
- **우리 프로젝트와의 관계:** 이 논문의 알고리즘은 저장소의 grover_min_ip.v 가 이미 절반쯤 구현하고 있는 바로 그 구조입니다(비교기 오라클 + 임계값 레지스터 + 외부 루프). 마지막 장에서 무엇을 더 붙여야 하는지 구체적으로 정리했습니다.

목차

1. 서론 — 왜 AUD가 검색 문제인가 (원문 p.1-2)
2. 무선 AUD 시스템 모델 (원문 p.2-3)
3. 양자 원리와 그로버 알고리즘 (원문 p.3-4)
4. AUD 전용 양자 알고리즘 — DHA, IIMSA, 하이브리드 (원문 p.4-5)
5. 시뮬레이션 설정과 구현 회로 (원문 p.5-7)
6. 성능 분석 결과 (원문 p.7-12)
7. 결론 (원문 p.12-13)

1. 서론 — 왜 AUD가 검색 문제인가

원문 p.1-2 (Abstract, Section I)

1.1 문제 배경

5G의 두 축인 **URLLC**(ultrareliable low latency communication)와 **mMTC**(massive machine-type communication)는 각각 “지연을 극단적으로 줄인다”와 “엄청나게 많은 단말을 붙인다”를 목표로 합니다. 그런데 이 둘은 서로 당깁니다. 단말이 많으면 자원을 나눠 쓸 규칙이 필요하고, 규칙을 협상하려면 핸드셰이크가 필요하고, 핸드셰이크는 곧 지연이니깐요.

LTE 시대의 해법은 자원을 **직교(orthogonal)** 하게 쪼개는 것이었지만, 단말 수가 많아지면 시그널링 오버헤드가 감당이 안 됩니다. 그래서 나온 것이 **NOMA**(nonorthogonal multiple access)입니다. 여러 단말이 같은 자원 블록에 **동시에, 비직교로** 접근합니다. 사용자 간 간섭(interuser interference)이 생기지만 스펙트럼 효율(spectral efficiency, SE)이 올라갑니다.

여기에 3GPP Release 16의 **grant-free(GF) 상향링크 접속**이 얹힙니다. 단말은 기지국(base station, BS)의 허가를 기다리지 않고 그냥 쏩니다. 지연은 사라지지만, **복잡도가 통째로 기지국 쪽으로 넘어갑니다**. 기지국은 여러 신호가 겹쳐진 중첩(superposition) 신호를 받아놓고, “이 안에 누가 들어 있나”를 스스로 알아내야 합니다. 이것이 AUD입니다.

1.2 고전 방법의 딜레마

- **ML(maximum likelihood)**: 최적 검출기입니다. 가능한 모든 사용자 활성 조합에 대해 우도(likelihood)를 계산하고 최선을 고릅니다. 사용자가 N 명이면 조합은 2^N 가지, 즉 $O(2^N)$ **완전탐색**입니다. 정확하지만 실용성이 없습니다.
- **준최적(suboptimal) 방법들**: PIC/SIC(병렬·순차 간섭 제거), MP(matching pursuit), AMP(approximate message passing) 등. 계산량은 줄지만 정확도가 떨어집니다.

즉 고전 수신기에서는 **성능과 실현 가능성 사이의 트레이드오프**를 피할 수 없습니다. 이 논문이 노리는 지점이 정확히 여기입니다.

참고: AUD를 “희소(sparse) 활성 벡터 복원” 문제로 보고 **압축센싱(compressed sensing)**으로 푸는 것이 이 분야의 흔한 접근이고, 위에 나온 MP·AMP가 바로 그 계열입니다. 하지만 이 논문은 압축센싱 쪽으로 가지 않습니다. **ML의 완전탐색 구조가 그로버의 구조와 그대로 겹치기 때문에**, 일부러 ML을 그대로 붙들고 갑니다. 이것이 이 논문을 “그로버 응용 논문”으로 읽어야 하는 이유입니다.

1.3 왜 그로버인가, 그리고 선행 연구와의 차이

그로버 알고리즘(Grover's algorithm)은 정렬되지 않은 데이터베이스에서 값을 $O(\sqrt{K})$ 에 찾습니다(K = 데이터베이스 크기, 고전은 $O(K)$). 그 확장이 **QMSA**(quantum minimum searching algorithm)

계열, 즉 최댓값이 아니라 **최솟값**을 찾는 알고리즘군입니다. 대표가 BBHT(Boyer-Brassard-Høyer-Tapp)와 DHA(Dürr-Høyer)입니다.

논문의 관찰은 간단합니다. **“ML의 수식 구조가 그로버의 구조와 완벽히 들어맞는다.”** ML은 결국 2^N 개 후보 중 거리가 최소인 것을 고르는 일이고, 그게 바로 최솟값 탐색이니깐요.

저자들이 정리한 선행 연구 지형:

선행 연구	내용	이 논문과의 차이
Botsinis et al. [9]	다중사용자 검출(MUD)에 BBHT/DHA 적용, 반복 상한 $22.5\sqrt{K/S}$	단순한 경우만 다
Habibie et al. [10], [11]	AUD에 원본 그로버 적용, 여러 코드 패밀리 비교	무잡음 가정
Norimoto & Ishikawa [12]	해의 개수 미지 상황에서 그로버로 최솟값 탐색	전력영역 NOMA
Xiang et al. [13]	개선된 QAOA	전력영역 NOMA
이 논문	코드영역 NOMA + 잡음 있는 ML + DHA 개선	—

핵심 공백: BBHT/DHA를 **코드영역 NOMA의 AUD**에 쓴 사례가 없었고, BBHT/DHA 자체도 여전히 복잡도가 높습니다.

1.4 논문의 기여

저자들이 명시한 기여는 두 가지입니다.

1. **IIMSA(improved iterative minimum searching algorithm)** 제안. DHA는 해의 개수 S 를 모른다고 가정하고 반복 횟수를 무작위로 뽑습니다. 하지만 통신 시스템은 그 통계적 거동을 **미리 오프라인으로 분석해 둘 수 있습니다**. 이 사전 지식으로 S 를 추정하면 그로버 반복 횟수를 정확히 잡을 수 있습니다.
2. **혼합 원리(mixed-principle) 전략**. DHA와 IIMSA의 **초기화**를 고전 방법(CCR, ZF)으로 수행합니다. 반복 횟수는 줄고, 정확도는 떨어지지 않습니다.

한 문장으로 요약하면, 저자들의 문제의식은 이렇습니다. 기존 양자 알고리즘은 범용 문제용으로 설계되어 **“초기화와 파라미터 설정이 눈감고(blindly) 이루어진다”** — 그런데 우리는 시스템에 대해 아는 게 있으니, 그걸 써서 눈을 뜨게 하자는 것입니다.

2. 무선 AUD 시스템 모델

원문 p.2-3 (Section II)

2.1 시스템 모델

설정은 이렇습니다.

- 단일 안테나 기지국 1대, 단말 N 대.
- 단말은 평소 슬립 모드, 메시지 보낼 때만 활성.

- **코드영역 NOMA(CD-NOMA):** 사용자마다 고유 코드로 구별. 코드워드 하나는 SF (spreading factor, 확산 계수) 개의 칩(chip)으로 구성. 코드북은 $\mathbf{C} \in \mathbb{R}^{N \times SF}$.
- $N \geq SF$ — 이게 NOMA의 정의입니다. 칩 수보다 사용자가 많으니 코드가 직교할 수 없고, 그래서 간섭이 생깁니다.
- 활성 상태는 $\mathbf{b} \in \{0, 1\}^N$ 로 모델링. $b_i = 1$ 이면 사용자 i 가 활성.
- 채널(channel)은 완벽하다고 가정($H = 1$, 전력 제어 루프로 보상됨). 잡음은 성분마다 독립인 AWGN, $\mathcal{N}(0, \sigma^2)$, $\mathbf{w} \in \mathbb{R}^{SF}$.

수신 신호는 (원문 Eq. 1):

$$\mathbf{y} = \mathbf{b} \cdot \mathbf{C} + \mathbf{w}$$

$\mathbf{y} \in \mathbb{R}^{SF}$ 는 활성 집합 $\mathbf{b}$, 코드북 $\mathbf{C}$, 잡음 $\mathbf{w}$ 세 가지에만 의존합니다. AUD의 목표는 $\mathbf{y}$ 와 $\mathbf{C}$ 로부터 $\mathbf{b}$ 를 되찾는 것입니다.

여기서 잠깐 — 이 한 줄이 이 논문 전체의 뼈대입니다. $\mathbf{b}$ 는 N 비트짜리 이진 벡터이고, 후보는 정확히 2^N 개입니다. 나중에 그로버의 인덱스 레지스터에 들어갈 $|x\rangle$ 가 바로 이 $\mathbf{b}$ 입니다. 즉 “데이터베이스의 x 번째 원소”와 “ x 번째 활성 조합”이 같은 것입니다.

세 가지 코드 패밀리를 씁니다.

1. **Unipolar:** $\mathbf{C} \in \{0, 1\}^{N \times SF}$
2. **Bipolar:** $\mathbf{C} \in \{-1, 1\}^{N \times SF}$
3. **Random Gaussian:** 각 성분이 가우시안 분포를 따르고, 사용자 간 전력을 맞추기 위해 크기로 정규화 $\rightarrow \mathbf{C}/\|\mathbf{C}\| \in \mathbb{R}^{SF}$

2.2 CCR — 관용 상관 수신기

CCR(conventional correlation receiver, 관용 상관 수신기)은 가장 단순한 검출기입니다. 사용자를 **하나씩 독립적으로** 판정합니다. 수신 신호 $\mathbf{y}$ 를 해당 사용자의 코드 $\mathbf{C}_i$ 와 상관시키고, 미리 정한 임계값 T 를 넘으면 활성으로 봅니다 (원문 Eq. 2):

$$\hat{b}_i^{CCR} = \begin{cases} 1, & \text{if } \sum_i \mathbf{y} \cdot \mathbf{C}_i \geq T \\ 0, & \text{otherwise} \end{cases}$$

전체 활성 집합을 추정하려면 이걸 N 번 돌려야 합니다.

2.3 ZF — 영강제 수신기

ZF(zero forcing, 영강제)는 CCR과 달리 **모든 사용자를 한꺼번에** 봅니다. 코드북 $\mathbf{C}$ 를 알고 있으니 역행렬을 곱해 풀면 됩니다. 다만 $\mathbf{C} \in \mathbb{R}^{N \times SF}$ 는 정사각이 아닌 직사각 행렬이고 계수(rank)가 SF 이므로, **의사역행렬(pseudo-inverse)** 을 씁니다:

$$\mathbf{c}_C = (\mathbf{C}^T \mathbf{C})^{-1} \mathbf{C}^T$$

추정 활성 집합은 (원문 Eq. 3):

$$\hat{\mathbf{b}}^{ZF} = \mathbf{y} \cdot \mathbf{c}_C$$

계산은 1번이면 끝입니다. (CCR은 N 번, ZF는 1번 — 뒤에서 하이브리드 비용을 따질 때 다시 나옵니다.)

2.4 ML — 최대우도, 그리고 병목

ML(maximum likelihood)은 AUD의 최적 검출기입니다. AWGN 채널이고 활성 확률이 균등하면, ML은 “기대 수신 신호와 실제 수신 신호 사이의 거리를 최소화하는 활성 집합”을 찾는 문제로 환원됩니다 (원문 Eq. 4):

$$\hat{b}^{ML} = \arg \min_{\{b_i\}_{i \in [N]}} \left\| \mathbf{y} - \sum_{i=1}^N b_i \cdot \mathbf{c}_i \right\|_2^2$$

이 식이 이 논문의 **목적함수**입니다. 앞으로 나올 모든 $f(x)$, δ , “거리”, “norm”은 전부 이 $\|\mathbf{y} - \mathbf{b} \cdot \mathbf{C}\|_2^2$ 를 가리킵니다.

문제는 복잡도입니다. 우도 계산 자체는 위처럼 단순한 거리로 줄일 수 있지만, 2^N 개 거리 중 최소를 찾는 일이 사용자 수에 지수적으로 의존합니다. 그래서 $O(2^N)$, 고전 프로세서로는 사용자가 늘면 손을 못 땁니다.

정리: 고전 수신기는 성능(ML)과 실현 가능성(CCR/ZF) 중 하나를 포기해야 합니다. 양자 쪽에서 이 트레이드오프를 깰 수 있는지가 다음 절의 주제입니다.

3. 양자 원리와 그로버 알고리즘

원문 p.3-4 (Section III)

이 절은 우리 스터디 문서 [01장·07장·09장](#)과 내용이 겹칩니다. 이미 아는 내용이면 **3.4절(BBHT)**로 건너뛰어도 됩니다.

3.1 양자 표기법

큐비트(qubit)는 디랙 표기로 (원문 Eq. 5):

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

$\alpha, \beta \in \mathbb{C}$ 는 확률 진폭(probability amplitude)이고 $|\alpha|^2 + |\beta|^2 = 1$ 을 만족합니다. 진폭이 복소수이므로 (원문 Eq. 6)처럼 각도로도 쓸 수 있습니다:

$$|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\varphi} \sin \frac{\theta}{2} |1\rangle, \quad 0 \leq \theta \leq \pi, 0 \leq \varphi \leq 2\pi$$

큐비트 2개면 상태 4개 (원문 Eq. 7):

$$|\psi\rangle = \alpha_1 |00\rangle + \alpha_2 |01\rangle + \alpha_3 |10\rangle + \alpha_4 |11\rangle, \quad \sum_i |\alpha_i|^2 = 1$$

중요한 제약: **우리는 진폭에 직접 접근할 수 없습니다.** 얻을 수 있는 정보는 오직 측정(measurement)뿐이고, 측정은 상태벡터를 측정 기저의 한 상태로 무작위 사영(projection)합니다. 각 상태는 자기 진폭의 크기 제공에 비례하는 확률로 나타납니다.

3.2 그로버 알고리즘

그로버는 두 함수의 반복입니다 (원문 Fig. 1, p.3 — 아다마르 벡 $\rightarrow$ 오라클 $\mathbf{O} \rightarrow H^{\otimes n} \rightarrow 2|0^n\rangle\langle 0^n| \rightarrow H^{\otimes n}$ 순서의 회로도).

1. **오라클(oracle) $\mathbf{O}$** — 조건을 만족하는 상태를 마킹(marked state)합니다.
2. **확산 연산자(diffuser) $\mathbf{D}$** — 마킹된 상태의 진폭을 증폭(amplitude amplification)합니다.

시작은 균등중첩입니다. 모든 큐비트를 $|0\rangle$ 로 두고 아다마르(Hadamard) $H^{\otimes N}$ 를 걸면 (원문 Eq. 8):

$$|\psi\rangle = \frac{1}{\sqrt{K}} \sum_{x=1}^K |x\rangle$$

여기서 $|x\rangle \in \{0, 1\}^N$ 는 인덱스이고 $K = 2^N$ 은 기저 상태의 개수입니다.

표기 주의: 논문은 N 을 사용자 수로도 쓰고 큐비트 수로도 씁니다. 이건 오타가 아니라 설계 그 자체입니다 — **잠재 사용자 1명당 큐비트 1개**를 배정하므로 두 N 이 같은 값입니다. 따라서 검색 공간 $K = 2^N$ 은 곧 “가능한 활성 조합의 수”입니다.

오라클은 표적 상태를 다음처럼 표시합니다 (원문 Eq. 9):

$$\mathbf{O}|x\rangle = \begin{cases} |x\rangle, & \text{if } f(x) = \delta \\ -|x\rangle, & \text{otherwise} \end{cases}$$

잠깐, 부호가 거꾸로 아닌가요? 우리가 7장에서 배운 표준 오라클은 정답에 -1 을 붙입니다. 그런데 이 식은 정답만 그대로 두고 나머지 전부에 -1 을 붙입니다. 둘은 전역 위상(global phase) -1 만큼만 다르고, 전역 위상은 측정 결과에 영향을 주지 않으므로 **물리적으로 같은 오라클**입니다. 헷갈리지만 틀린 건 아닙니다. 다만 하드웨어로 옮길 때는 표준 형태(정답에만 -1)를 쓰는 게 훨씬 쉽습니다 — 마킹 대상만 건드리면 되니까요.

확산 연산자는 평균 반사(inversion about the mean)를 수행하는 유니터리(unitary)입니다 (원문 Eq. 10):

$$\mathbf{D} = H^{\otimes N}(2|0^N\rangle\langle 0^N| - I)H^{\otimes N} = 2|\psi\rangle\langle\psi| - I$$

그로버 연산자 $\mathcal{G}$ 전체는 (원문 Eq. 11):

$$\mathcal{G}|\psi\rangle = \mathbf{D.O.}|x\rangle = H^{\otimes N}(2|0^N\rangle\langle 0^N| - I)H^{\otimes N}.\mathbf{O.}|x\rangle$$

한 번 돌리면 원하는 해가 나올 확률이 올라가지만, **원치 않는 해가 나올 가능성도 여전히 남습니다**. 이 잔여 확률이 뒤의 알고리즘 설계를 지배합니다.

3.3 최적 반복 횟수 — 해의 개수를 알 때

반복 횟수는 데이터베이스 크기 K 와 해의 개수 S 에 달려 있습니다. 최적값은:

$$L_{\text{opt}} = \left\lfloor \frac{\pi}{4} \sqrt{K/S} \right\rfloor$$

($\lfloor \cdot \rfloor$ 는 바닥 함수. 원문은 Nielsen & Chuang [18]을 인용합니다.)

3.4 최적 반복 횟수 — 해의 개수를 모를 때 (BBHT)

현실에서는 S 를 모르는 경우가 많고, 그러면 위의 L_{opt} 를 계산할 수 없습니다. BBHT는 이 상황을 **사전 지식 없이** 처리합니다. 크기 K 의 데이터베이스 U 에서 값 δ 를 찾는 게 목표입니다.

Algorithm 1 — BBHT (원문 p.4)

- 입력: K, m, λ / 출력: x_{out}
- 1. $m \leftarrow 1, \lambda \leftarrow \frac{6}{5}$ (또는 $\lambda \leftarrow \frac{4}{3}$), $L_{\text{BBHT}} \leftarrow 1$
- 2. **while** $L_{\text{BBHT}} < 4.5\sqrt{K}$ **do**
- 3. j 를 $\{0, \dots, \lfloor m \rfloor\}$ 에서 균등하게 뽑는다
- 4. 균등중첩 $|\psi\rangle = \frac{1}{\sqrt{K}} \sum_{x=0}^{K-1} |x\rangle$ 를 준비한다 (Eq. 8)
- 5. 그로버를 j 번 적용: $|\psi_{\text{out}}\rangle = \mathcal{G}^j|\psi\rangle$
- 6. $L_{\text{BBHT}} \leftarrow L_{\text{BBHT}} + j$
- 7. $|\psi_{\text{out}}\rangle$ 를 측정해 x_{out} 을 얻는다
- 8. **if** $U[x_{\text{out}}] = \delta$ **then** 해를 찾았으므로 while 루프를 빠져나온다
- 9. **else** $m \leftarrow \min(\lambda m, \sqrt{K})$
- 10. **end**

아이디어는 단순합니다. S 를 모르니 L_{opt} 를 못 정한다 $\rightarrow$ 그러면 **반복 횟수를 $\{0, \dots, \lfloor m \rfloor\}$ 에서 무작위로 뽑아 보고**, 실패하면 m 을 λ 배씩 키워 탐색 범위를 넓힌다. m 은 $\sqrt{K}$ 에서 멈춥니다.

BBHT의 복잡도는 $\frac{9}{2}\sqrt{K}$ 로 상계가 증명되어 있습니다. 알고리즘은 $U[x_{\text{out}}] = \delta$ 가 되거나 L_{BBHT} 가 이 상계에 닿으면 멈춥니다.

4. AUD 전용 양자 알고리즘 — DHA, IIMSA, 하이브리드

원문 p.4-5 (Section IV)

4.1 자명한 조건 vs 비자명한 조건 — 이 절의 출발점

이 절의 논리적 출발점은 짚고 넘어갈 가치가 있습니다.

그로버와 BBHT에서 하는 **자명한(trivial) 조건**을 만족합니다. 즉 고전 계산 한 번이면 확인되는 조건입니다 (예: $f(x) < \delta$ 인지 판정 — δ 만 알면 바로 확인 가능). 그런데 **ML은 이 부류가 아닙니다**. “이게 최솟값인가?”는 다른 모든 후보를 봐야 답할 수 있으니까요.

그래서 한 단계 더 들어갑니다: **최솟값 찾기 같은 비자명한(nontrivial) 조건**을 다뤄야 합니다. 이때 해법은 문제를 **자명한 조건의 반복**으로 바꾸는 것입니다 — “ δ 보다 작은 것을 찾아라”는 자명하니, δ 를 계속 낮춰 가면 최솟값에 수렴합니다. 이것이 DHA와 IIMSA를 관통하는 아이디어이고, 그로버를 실전 문제에 붙이는 가장 중요한 기술입니다.

그리고 앞 절에서 본 대로, 해의 개수 S 를 아느냐 모르느냐에 따라 접근이 갈립니다. 논문은 두 알고리즘을 제시합니다: **모르는 경우(DHA)** 와 **아는 경우(IIMSA)**.

4.2 DHA (Dürr-Høyer Algorithm)

여기서는 데이터 구조에 대한 사전 정보가 전혀 없다고 가정합니다. 즉 **눈감고(blind)** 탐색해야 합니다.

최솟값 찾기의 고전적 직관은 이렇습니다. 초기값 하나를 고른다 → 그보다 작은 값을 찾는다 → 찾으면 기준을 그걸로 갱신하고 반복 → 더 작은 값이 안 나오면 그게 최솟값. DHA는 이 루프의 “**그보다 작은 값을 찾는다**” 부분을 그로버로 대체합니다.

DHA는 최솟값의 인덱스를 **확률 1/2 이상**으로, $O(\sqrt{K})$ 시간에 찾습니다.

Algorithm 2 — DHA (원문 p.5)

- 입력: K, m, λ / 출력: x_{out}
- 1. $m \leftarrow 1, \lambda \leftarrow \frac{6}{5}$ (또는 $\lambda \leftarrow \frac{4}{3}$), $L_{\text{DHA}} \leftarrow 1$
- 2. **while** $L_{\text{DHA}} < 22.5\sqrt{K}$ **do**
- 3. $0 \leq i_x \leq K - 1$ 을 균등분포로 뽑는다 (기준 인덱스 초기화)
- 4. 반복 횟수 j 를 $\{0, \dots, \lfloor m \rfloor\}$ 에서 균등하게 뽑는다
- 5. 균등중첩 $|\psi\rangle = \frac{1}{\sqrt{K}} \sum_{x=0}^{K-1} |x\rangle$ 를 준비한다
- 6. 그로버 연산자 $\mathcal{G}$ 를 j 번 적용해 $|\psi_{\text{out}}\rangle = \mathcal{G}^j |\psi\rangle$. 이때 $U[x] < U[i_x]$ 인 상태를 마킹한다
- 7. $|\psi_{\text{out}}\rangle$ 를 측정해 x_{out} 을 얻는다
- 8. $L_{\text{BBHT}} \leftarrow L_{\text{BBHT}} + j$

9. $L_{\text{DHA}} \leftarrow L_{\text{DHA}} + L_{\text{BBHT}}$
10. **if** $U[x_{\text{out}}] < U[i_x]$ **then** $i_x \leftarrow x_{\text{out}}$ 하고 while 루프를 빠져나온다
11. **else** Step 3으로 돌아간다
12. **end**

Step 6이 이 논문의 심장입니다. 오라클의 조건이 “인덱스가 정답과 같은가”가 아니라 $**U[x]$ 가 현재 기준 $U[i_x]$ 보다 작은가 $**$ 라는 **비교**로 바뀌었습니다. 정답을 몰라도 오라클을 만들 수 있게 되는 순간입니다.

탐색 절차의 복잡도는 $22.5\sqrt{K}$ 로 주어집니다.

의사코드 읽을 때 주의: Step 10의 “exit the while loop”는 본문 설명(“더 작은 값이 안 나올 때까지 반복적으로 갱신”)과 어긋나 보입니다. 의도는 “ i_x 를 갱신하고 **바깥 루프를 계속 돈다**”에 가깝습니다. 또 Algorithm 2에는 BBHT의 $m \leftarrow \min(\lambda m, \sqrt{K})$ 갱신 줄이 빠져 있는데, 입력에 λ 가 있는 걸 보면 실제로는 BBHT처럼 m 을 키워 가는 것으로 읽는 게 자연스럽습니다. 구현할 때는 본문 서술을 기준으로 삼으세요.

4.3 IIMSA — 해의 개수를 추정해서 반복 횟수를 정확히

DHA는 S 를 모른다고 가정합니다. 하지만 통신 시스템은 **사전 오프라인 연구**로 그 통계적 거동을 알아낼 수 있습니다. 잡음 분산, 코드북, 활성 확률이 주어지면 “거리가 $\hat{\delta}$ 미만인 후보 집합이 평균 몇 개인가”는 통계적으로 계산 가능한 양입니다.

S 를 알면 반복 횟수를 무작위로 뽑을 이유가 없습니다. 그냥 L_{opt} 를 쓰면 됩니다. 이것이 IIMSA입니다.

Algorithm 3 — IIMSA (원문 p.5)

- 입력: $\mathbf{C}, K, L_{\text{IMSA}}, \hat{\delta}$ / 출력: $\delta_x, \mathbf{b}$
1. $i \leftarrow (0, K - 1)$ 에서 균등하게 뽑는다
 2. $\hat{\delta}(i) \leftarrow i$ 에 대응하는 값으로 설정 (그 후보 집합과 수신 신호의 거리)
 3. $\hat{S} \leftarrow \|\mathbf{y} - \mathbf{b} \cdot \mathbf{C}\|_2^2 \leq \hat{\delta}$ (거리가 $\hat{\delta}$ 이하인 집합의 **개수** — 통계 분석으로 추정)
 4. $nb_{it} \leftarrow 1$
 5. **while** $\hat{S} \neq 0$ 또는 $nb_{it} \leq L_{\text{IMSA}}$ **do**
 6. $\mathbf{b} \leftarrow \mathcal{G}(L_{\text{opt}})$ (그로버를 L_{opt} 번 돌리고 측정)
 7. $\delta_x \leftarrow \|\mathbf{y} - \mathbf{b} \cdot \mathbf{C}\|_2^2$
 8. $\hat{S} \leftarrow \|\mathbf{y} - \mathbf{b} \cdot \mathbf{C}\|_2^2 \leq \delta_x$ (새 임계값 기준으로 해의 개수 재추정)
 9. $nb_{it} \leftarrow nb_{it} + L_{\text{opt}}$
 10. **if** $\delta_x < \hat{\delta}$ **then** $\hat{\delta} \leftarrow \delta_x$
 11. **end**

동작을 말로 풀면:

1. 인덱스 i 를 $[0, K - 1]$ 에서 무작위로 고르고, 그 집합과 수신 신호의 거리를 계산해 $\hat{\delta}$ 를 초기화합니다.
2. 통계 분석으로 **해의 개수** S (= 거리가 현재 $\hat{\delta}$ 보다 작은 집합의 수)를 추정합니다.
3. $S = 0$ **이면** 현재 $\hat{\delta}$ 가 곧 최소 거리입니다. 끝.

4. 아니면 $f(x) < \hat{\delta}$ 를 찾는 그로버 $\mathcal{G}$ 를 돌립니다. 출력은 새 활성 집합이고, 거기서 새 δ_x 와 새 S 가 나옵니다.
5. $\delta_x < \hat{\delta}$ 이면(측정이 유효한 해를 반환했다면) $\hat{\delta}$ 를 δ_x 로 갱신하고 다시 $\mathcal{G}$. 다음 반복에서는 해집합이 더 작아져 있습니다.
6. $S = 0$ 이 될 때까지 반복.

각 그로버 호출은 아래 횟수만큼 수정된 오라클을 돌립니다:

$$L_{\text{opt}} = \left\lfloor \frac{\pi}{4} \sqrt{2^N / S} \right\rfloor$$

중요한 경고 두 가지를 저자들이 붙입니다.

- S 가 데이터베이스 크기의 절반보다 크면, 진폭증폭이 오히려 원치 않는 경우를 증폭해서 탐색을 엉뚱한 방향으로 밀어붙입니다. ($S/K > 1/2$ 이면 그로버 회전이 과회전 영역으로 들어갑니다.)
- 그래서, 그리고 반복 횟수 제한이 있는 DHA와 조건을 맞추기 위해, 실패 시도 횟수를 L_{IIMSA} 로 제한합니다.

의사코드 주의: Step 5의 while $\hat{S} \neq 0$ or $\text{nb_it} \leq L_{\text{IIMSA}}$ 는 논리적으로 and 여야 자연스럽습니다(“해가 남아 있고, 예산도 남아 있는 동안”). 본문 서술(“ $S = 0$ 이 될 때까지 반복하되 L_{IIMSA} 로 제한”)을 기준으로 읽으세요.

DHA vs IIMSA 한눈 비교

	DHA	IIMSA
해의 개수 S	모름 (가정)	시스템 통계로 추정
회당 그로버 반복	$j \sim \text{Uniform}\{0, \dots, \lfloor m \rfloor\}$, 실패하면 m 확대	$L_{\text{opt}} = \lfloor \frac{\pi}{4} \sqrt{2^N / S} \rfloor$ 로 확정
종료 조건	더 작은 값을 못 찾음 / 예산 소진	$\hat{S} = 0$ / 예산 L_{IIMSA} 소진
예산 상계	$22.5\sqrt{K}$	L_{IIMSA} (비교를 위해 $22.5\sqrt{2^N}$ 로 설정)
장점	사전 지식 불필요	헛돌지 않음
단점	반복이 자주 낭비됨	오프라인 통계 분석 필요

4.4 양자-고전 하이브리드 — CCR/ZF로 초기화

마지막 개선입니다. 수신 신호를 고전 전처리해서 얻는 지식을 활용합니다. 목적은 두 가지입니다.

- 양자 탐색을 정확히 초기화해서, 해의 개수가 너무 많아지는 상황(위의 $S > K/2$ 문제)을 예방합니다.
- 표적 해 δ 에 더 가까운 데서 출발하니 탐색이 빨라집니다.

방법은 2.2·2.3절의 CCR과 ZF를 수신 신호 $\mathbf{y}$ 에 적용해 활성 상태를 추정하고, 그 기준 집합과 $\mathbf{y}$ 의 거리를 계산해 각각 δ_{CCR} , δ_{ZF} 를 얻는 것입니다. 이걸 $\hat{\delta}$ 의 초기값으로 씁니다.

결합 알고리즘은 총 4가지: CCR-DHA, ZF-DHA, CCR-IIMSA, ZF-IIMSA. 원본 DHA·IIMSA까지 하면 비교 대상은 6개 검출기입니다.

비용 관점에서 저자들이 정직하게 짚습니다: 이 접근은 **고전 계산을 더 요구합니다**(CCR N 번, ZF 1번). 대신 **양자 복잡도를 크게 줄일 것으로 기대합니다**. 지금 하드웨어 현실에서 큐비트 연산이 고전 곱셈보다 훨씬 비싸다는 걸 생각하면 합리적인 거래입니다.

5. 시뮬레이션 설정과 구현 회로

원문 p.5-7 (Section V-A, V-B, V-C)

5.1 전체 구성

처리 흐름은 (원문 Fig. 2, p.6)의 순서도로 정리됩니다. Start $\rightarrow \mathbf{y} = \mathbf{b} \cdot \mathbf{C} + \mathbf{w}$ 생성 $\rightarrow$ **양자화(Quantization)** $\rightarrow$ **전처리(CCR 또는 ZF, 또는 생략)** $\rightarrow$ **최솟값 탐색(IIMSA 또는 DHA)** $\rightarrow \hat{\mathbf{b}} \rightarrow$ End. 각 단계에서 두 갈래 중 하나를 고르는 구조라, 6가지 검출기 조합이 이 하나의 그림에서 나옵니다.

시뮬레이션은 **MATLAB**으로 했습니다. 이유가 우리에게 흥미롭습니다 — “**이 회로들에 필요한 큐비트 수가 32개를 넘기 때문**” 입니다. 즉 실제 양자 하드웨어나 상용 시뮬레이터로는 못 돌려서 직접 구현했다는 뜻이고, 우리가 만들려는 **FPGA 에뮬레이터가 정확히 이 자리를 노립니다**.

평가 지표는 두 개입니다.

- P_s (성공확률): 모든 활성 사용자를 정확히 검출하고, 비활성 사용자를 활성으로 오판하지 않은 경우, 즉 $\hat{\mathbf{b}} = \mathbf{b}$ 인 비율. 부분 점수 없는 전부-아니면-전무 기준입니다.
- nb_{it} (복잡도): 그로버 반복의 총 횟수. L_{DHA} / L_{IIMSA} 와 동일한 누적 방식입니다.

최적은 P_s 는 높고 nb_{it} 은 작은 것입니다. 저자들은 고전 프로세서로 시뮬레이션 가능하도록 **작은 코드 패밀리**만 다뤘고, 이를 개념 증명(proof of concept)이라 부르며 더 큰 크기로 확장 가능하다고 밝힙니다.

표 1 — 코드 패밀리별 SF와 사용자 수 N (원문 Table I, p.6 그대로)

코드 타입	SF	사용자 수 N	검색 공간 $K = 2^N$
Unipolar	3	4	16
Unipolar	4	5	32
Unipolar	5	7	128
Unipolar	6	9	512
Unipolar	7	11	2048
Bipolar	3	3	8
Bipolar	4	5	32
Bipolar	5	6	64
Bipolar	6	8	256
Bipolar	7	10	1024
Gaussian	3	5	32
Gaussian	4	6	64
Gaussian	5	7	128

코드 타입	SF	사용자 수 N	검색 공간 $K = 2^N$
Gaussian	6	8	256
Gaussian	7	9	512

(K 열은 원문에 없고, 검색 공간 크기를 감 잡으라고 제가 덧붙였습니다.)

수행한 분석은 네 가지입니다: (1) IIMSA vs DHA, (2) DHA·CCR-DHA·ZF-DHA, (3) IIMSA·CCR-IIMSA·ZF-IIMSA, (4) $L_{\max}$ 에 따른 IIMSA vs DHA. 여기에 (5) 잡음 있는 양자 하드웨어 분석이 추가됩니다.

5.2 구현 회로 — 이 논문에서 가장 중요한 절

원문 p.6-7 (Section V-B). 하드웨어를 만들 사람이라면 이 절만은 원문 그림과 함께 보세요.

5.2.1 네 개의 레지스터 그로버 회로(원문 Fig. 3, p.7)는 레지스터 4개로 구성됩니다. 그림은 $N = 3$ 큐비트 예시이고, 기저 상태 8개($|000\rangle \sim |111\rangle$)를 다룹니다.

레지스터	Fig. 3의 큐비트	크기	역할
Index	$ b_2\rangle, b_1\rangle, b_0\rangle$	N 큐비트 (사용자 1명당 1개)	함수 $f(x)$ 의 인자 x — 곧 후보 활성 벡터 $\mathbf{b}$. $H^{\otimes N}$ 의
Reference	$ a_2\rangle, a_1\rangle, a_0\rangle$	Z 큐비트	비교 대상인 표적값 δ (임계값)
Ancilla	$ g_1\rangle, g_0\rangle$	수 큐비트	부분 결과 저장. 연산의 유니터리성을 보장하기 위해 필
Mark	$ r\rangle$	1 큐비트	X 후 H 로 $ -\rangle$ 준비. Eq. 9의 음수 부호를 공급

Z 는 $|f(x)\rangle$ 를 표현하는 데 필요한 최소 큐비트 수이고 목표 정밀도와 연관됩니다(5.2.3절). Mark 레지스터는 $|-\rangle$ 상태라, 여기로 CNOT을 쓰면 위상 킥백(phase kickback) 으로 인덱스 레지스터에 -1 이 얹힙니다 — 우리 7장에서 배운 그 메커니즘 그대로입니다.

참고로 저자들의 이전 연구 [10], [11]에서는 $|f(x)\rangle = |\delta\rangle$ 인 더 단순한 회로를 다뤘습니다 — 즉 f 가 항등함수라 “인덱스가 정답이냐”만 보면 됐습니다. 이번 논문이 그보다 한 단계 어려운 이유가 바로 f 가 진짜 계산이 되었기 때문입니다.

5.2.2 비교기를 오라클로 바꾸기 — 핵심 메커니즘 DHA처럼 최솟값을 찾으려면 Eq. 9의 오라클을 재설계해서 $f(x) < \delta$ 를 만족하는 상태를 마킹해야 합니다. 방법은 놀랄 만큼 고전 하드웨어스럽습니다.

두 상태 $|a\rangle$ 와 $|b\rangle$ 가 인코딩한 값의 차 ($a - b$) 를 계산합니다. 그러면 부호로부터 대소를 알 수 있고, $r = \text{MSB}(a - b)$ 를 자리올림(carry)처럼 씁니다 (원문 Eq. 12):

$$r = \begin{cases} 1, & a > b \\ 0, & \text{otherwise} \end{cases}$$

그리고 이 r 을 Mark 레지스터의 $|-\rangle$ 에 곱해 오라클에 음수 부호를 공급합니다. 이 논문에서 $|a\rangle$ 와 $|b\rangle$ 는 각각 $|\delta\rangle$ 와 $|f(x)\rangle$ 에 대응합니다 — Eq. 9의 오라클 이론과 맞추기 위해서입니다.

한 줄 요약: 오라클 = 감산기 + MSB 추출 + CNOT-to- $|-\rangle$. 정답을 몰라도 됩니다. 아는 숫자 δ 와 계산한 값 $f(x)$ 를 빼서 부호만 보면 되니까요. 이것이 그로버를 실제 문제에 붙이는 방법이고, 이 논문 전체가 이 한 가지 트릭 위에 서 있습니다.

구현 시 부호 주의 — 여기서 한 번은 반드시 해맵니다. 원문 본문은 “차의 부호가 음수이면 $a < b$ 임을 알 수 있다. 따라서 $r = \text{MSB}(a - b)$ ” 라고 쓰는데, 이는 $\text{MSB}(a - b) = 1 \iff a < b$ 를 뜻합니다. 그런데 바로 이어지는 Eq. 12는 $r = 1 \iff a > b$ 라고 합니다. 둘은 서로 반대입니다.

어느 쪽이 맞을까요? $|a\rangle = |\delta\rangle, |b\rangle = |f(x)\rangle$ 이고 우리가 마킹하려는 조건은 $f(x) < \delta$, 즉 $b < a$, 즉 $a > b$ 입니다. 그러니 **Eq. 12(마킹 조건)가 의도한 것이 맞고**, $r = \text{MSB}(a - b)$ 쪽이 부호가 뒤집힌 서술입니다. 실제로 필요한 건 $r = \overline{\text{MSB}(a - b)}$ 또는 $r = \text{MSB}(b - a)$ 입니다. Verilog로 옮길 때 이 반전 하나를 놓치면 알고리즘이 **최댓값을 찾습니다** — 그리고 조용히 동작하기 때문에 디버깅이 오래 걸립니다.

5.2.3 양자화 — 실수를 큐비트에 담기 수신 신호 y 의 성분은 실수 $y \in \mathbb{R}^+$ 입니다. 그런데 이 논문은 **디지털 양자 처리**를 다루지 아날로그 양자 계산을 다루지 않습니다. 따라서 y 를 양자화해 y_q 를 만들어야 합니다.

- **정수부:** $2^I - 1$ 로 상계. I 는 $|f(x)\rangle$ 가 가질 수 있는 값의 전 범위를 커버하는 데 필요한 큐비트 수.
- **소수부:** 정밀도 지표 $p \in \mathbb{N}$ 로 정의. p 가 클수록 정확하지만 메모리가 커집니다.
- **레지스터 크기:** $Z = I + p$.

Reference 레지스터에 넣는 양자화 값은 (원문 Eq. 13):

$$y_q = \min \left(\max \left(0, \frac{\text{round}(y \times 2^p)}{2^p} \right), 2^{I-1} \right)$$

이 논문은 소수부 정밀도로 $p = 2$ 를 씁니다.

이 식은 우리에게 전혀 낯설지 않습니다. $\text{round}(y \times 2^p) / 2^p$ 는 **고정소수점 양자화**이고, $\min(\max(0, \cdot), 2^{I-1})$ 은 **포화(saturation) 클리핑**입니다. grover_ip.v 의 Q1.16, grover_min_ip.v 의 Q4.16이 하는 일과 정확히 같은 종류입니다. 양자 회로 논문인데 “정밀도 p 를 올리면 정확하지만 메모리가 커진다”는 트레이드오프를 이야기하는 게 흥미롭죠 — 큐비트를 비트로 읽으면 그냥 하드웨어 설계 이야기입니다.

5.2.4 오라클 내부 — 네 개의 블록 y_q 를 얻은 뒤, 그로버 탐색 회로를 (원문 Fig. 4, p.7)의 $\mathcal{G}$ 로 수정합니다. 전처리된 y 와 δ 가 회로로 들어갑니다. 회로는 네 블록으로 구성됩니다.

블록	이름	하는 일
A_f	adder function (덧셈 함수)	$y - b.C$ 를 계산
N_f	norm function (노름 함수)	노름 $\ y - b.C\ _2^2$ 를 계산
U_δ	오라클 부분	노름이 δ 보다 작은 상태($f(b) < \delta$)를 식별
M_f	마킹	앞에서 식별된 상태를 마킹

그리고 결정적으로: 이 블록들은 역순으로 한 번 더 반복되어 언컴퓨테이션(uncomputation)을 수행하고, 그 다음에야 확산 연산자로 해의 진폭을 증폭합니다. Fig. 4의 실제 순서는 $A_f \rightarrow N_f \rightarrow U_\delta \rightarrow M_f \rightarrow U_\delta \rightarrow N_f \rightarrow A_f \rightarrow D \rightarrow$ 측정입니다.

왜 언컴퓨테이션이 필요한가: 중간 계산 결과가 Ancilla에 남아 있으면 인덱스 레지스터와 얽혀(entangled) 있어서, 확산 연산자의 평균 반사가 제대로 작동하지 않습니다. 양자에서는 “쓰레기 값”을 그냥 버릴 수 없고 반드시 되돌려 놔야 합니다. 이것이 양자 회로가 고전 회로보다 대략 2배 깊어지는 이유이고, Ancilla 레지스터가 존재하는 이유입니다.

여기서 중요한 사실 하나 — 이 회로에는 데이터베이스가 저장되어 있지 않습니다. $U[x]$ 는 메모리에서 읽어 오는 값이 아니라, A_f 와 N_f 가 매 인덱스마다 즉석에서 계산하는 값입니다. 중첩 덕분에 2^N 개 후보 전부에 대해 이 계산이 한 번에 일어납니다. 그로버 응용에서 “데이터베이스 검색”이라는 비유가 오해를 부르는 지점이 정확히 여기입니다 — 실제로는 함수 역산(function inversion) 입니다.

5.3 코드 패밀리별 구현

원문 p.7 (Section V-C)

1) Unipolar Code — $C \in \{0, 1\}^{N \times SF}$. 코드워드는 패밀리 내 하나 이상의 코드와 **비직교가 되도록** 무작위 선택합니다(NOMA이므로 직교하면 안 됩니다). 원문 예시가 이해에 좋습니다. $N = 4, SF = 3$ 일 때 $c_1 = [1, 1, 0], c_2 = [0, 1, 1], c_3 = [0, 0, 1], c_4 = [1, 0, 1]$ 이면 무잡음 기대 신호는 (원문 Eq. 14):

$$f(b_1, b_2, b_3, b_4) = \begin{bmatrix} b_1 + b_4 \\ b_1 + b_2 \\ b_2 + b_3 + b_4 \end{bmatrix}$$

즉 오라클이 계산해야 할 $b \cdot C$ 는 **활성 사용자들의 코드워드를 칩별로 더한 것**입니다. 이 예에서 그로버 회로가 필요로 하는 산술은 고작 3개 칩에 대한 덧셈 몇 번입니다.

2) Bipolar Code — $C \in \{-1, 1\}^{N \times SF}$. 음수 계산은 1의 보수(one's complement), 즉 비트 반전으로 처리합니다. Bipolar 코드는 두 unipolar 코드를 더해서 만드는데 그중 하나를 반전시킵니다. 음수 값 계산을 위해 양자 연산을 적응시킨 내용은 저자들의 선행 연구 [11]에 있습니다.

3) Random Gaussian Code — 각 성분이 가우시안 분포로 무작위 생성됩니다. 사용자 간 전력을 같게 하기 위해 크기로 정규화하므로 실제 코드는 $C/\|C\|$ 입니다. 양자 회로에 올리려면 소수를 다뤄야 하고, 정밀도 $p = 2$ 의 이진 소수 표현으로 해를 식별합니다.

왜 코드 패밀리가 성능을 가르는가: 결과 절 내내 반복되는 설명은 성분 간 유클리드 거리(Euclidean distance) 입니다. Bipolar는 $\{-1, 1\}$ 이라 성분 간 거리가 크고(두 값의 차가 2), Unipolar는 $\{0, 1\}$ 이라 거리가 절반(차가 1)이며, Gaussian은 정규화된 실수라 성분들이 서로 가깝습니다. 거리가 멀수록 잡음에 덜 흔들리고 검출이 정확합니다. 이게 아래 모든 그래프의 순서(Bipolar > Unipolar > Gaussian)를 설명합니다.

6. 성능 분석 결과

원문 p.7-12 (Section V-D)

6.1 IIMSA vs DHA — 잡음 분산에 따른 비교

원문 p.7-8 (Section V-D1), 원문 Fig. 5, p.8

설정: L_{IIMSA} 와 L_{DHA} 둘 다 $22.5\sqrt{2^N}$ 로 상계. 잡음 분산 σ^2 를 10^{-2} 부터 10^{-1} 까지 훑습니다.

Fig. 5(a)-(c) — 성공확률: 세 코드 타입 모두 σ^2 가 커질수록 P_s 가 떨어집니다(당연합니다 — 잡음이 검출을 방해하니까요). SF마다 비직교 정도가 달라서 추세가 갈리는데, 흥미롭게도 $SF = 4$ 가 가장 높은 P_s 를 보이고 $SF = 3$ 이 뒤를 잇습니다. $SF = 4$ 의 코드가 상대적으로 비직교 성분이 적기 때문이라고 저자들은 설명합니다. P_s 의 절대 수준은 코드 타입이 지배합니다: 최악 σ^2 에서 Bipolar는 최저 **0.96**, Unipolar는 **0.7**, Gaussian은 **0.4** 입니다. 유클리드 거리 순서 그대로입니다.

IIMSA와 DHA는 작은 σ^2 영역에서는 거의 같습니다. 하지만 Bipolar·Unipolar에서 SF가 6이나 7로 커지면 IIMSA가 DHA를 앞섭니다. 즉 제안 알고리즘은 잡음이 크고 사용자·슬롯이 많은 어려운 영역에서 진가를 발휘합니다.

Fig. 5(d)-(f) — 그로버 반복 횟수(nb_{it} , 작을수록 좋음): 로그 축에 SF별로 IIMSA(파랑)/DHA(주황) 쌍이 층층이 놓입니다. 모든 SF, 모든 σ^2 , 모든 코드 타입에서 IIMSA가 DHA보다 아래에 있습니다. SF가 커질수록 격차가 벌어집니다.

여기서 처음 보면 의아한 현상이 하나 있습니다: nb_{it} 이 σ^2 에 거의 무관하게 평평합니다. 저자들의 설명이 좋습니다 — nb_{it} 은 “가장 가까운 활성 집합으로 수렴하는 속도”를 나타내는 양입니다. 잡음이 커지면 틀린 집합으로 수렴할 확률이 올라갈 뿐, 수렴 속도 자체는 별로 안 변합니다. 그래서 P_s 는 σ^2 에 민감하고 nb_{it} 은 둔감합니다. (P_s 와 nb_{it} 이 서로 다른 것을 재고 있다는 뜻이고, 두 지표를 같이 봐야 하는 이유입니다.)

코드 타입별 nb_{it} 도 흥미롭습니다. Gaussian이 복잡도가 가장 작습니다 — 성분 간 유클리드 거리가 작아서 “ δ 보다 작은” 후보가 금방 잡히기 때문입니다(대신 그만큼 틀리기도 쉬워서 P_s 가 낮습니다). Bipolar는 거리가 큰데도 낮은 nb_{it} 로 수렴하고, Unipolar는 $SF = 7$ 에서 nb_{it} 이 가장 큼니다.

결론: 제안 알고리즘 IIMSA가 P_s 와 nb_{it} 양쪽 모두에서 원본 DHA를 능가합니다.

6.2 DHA · CCR-DHA · ZF-DHA

원문 p.8-10 (Section V-D2), 원문 Fig. 6, p.9

설정: $L_{\text{DHA}} = 22.5\sqrt{2^N}$, 잡음 분산 $\sigma^2 = 0.02$ 고정. x 축이 SF로 바뀝니다.

Fig. 6(a)-(c) — 성공확률. SF가 작을 때는 세 알고리즘이 비슷합니다. $SF > 6$ 부터 ZF-DHA가 나머지를 앞섭니다. 그리고 눈에 띄는 비대칭이 있습니다: CCR은 Unipolar에서 성능을 떨어뜨리고, Bipolar·Gaussian에서는 개선합니다.

이유가 깔끔합니다. Unipolar 코드는 성분이 $\{0, 1\}$ 이라 모든 칩에서 간섭이 양수로만 누적됩니다. 그래서 상관값이 임계값을 쉽게 넘고, CCR이 자꾸 “활성”이라고 오판합니다(오경보). 반면 Bipolar·Gaussian은 양·음의 간섭이 섞여 서로 상쇄되므로 오경보 위험이 줄어듭니다.

Gaussian의 P_s 가 전반적으로 낮은 것은 성분이 서로 가까워 잡음에 취약해서입니다. 다만 Gaussian은 곡선 모양이 다릅니다(Unipolar/Bipolar는 SF가 커지면 떨어지는데 Gaussian은 오르다가 완만해집니다). 두 효과가 겨루기 때문입니다 — SF가 커지면 두 코드가 혼동될 확률이 줄지만(개선), 사용자 수가 늘면 무작위 생성 때문에 가까운 코드워드 쌍이 나올 수 있습니다(악화). 곡선 모양이 이 절충을 보여 줍니다.

Fig. 6(d)-(f) — 반복 횟수. DHA가 모든 코드 타입에서 일관되게 가장 많은 반복을 요구하고, ZF-DHA가 가장 적습니다. ZF의 정확한 검출이 탐색을 더 일찍 멈추게 해 주기 때문입니다. SF가 작을 때(특히 $SF = 3$)는 세 알고리즘의 nb_{it} 이 거의 붙어 있는데, SF가 작으면 각 단계의 그로버 반복 수 자체가 작아서 비교 그래프가 촘촘해지기 때문입니다. 그리고 Unipolar에서는 CCR-DHA가 SF가 클 때 오히려 DHA보다 비효율적입니다 — 위에서 본 CCR의 Unipolar 간섭 문제 그대로입니다.

표 2 — Fig. 6 읽은 값 ($\sigma^2 = 0.02$)

아래 수치는 원문에 표로 실린 것이 아니라 제가 원문 그래프에서 눈으로 읽은 근삿값입니다. 추세를 잡는 용도로만 쓰고, 정확한 값이 필요하면 원문 그림을 직접 보세요.

코드	SF	N	P_s DHA	P_s CCR-DHA	P_s ZF-DHA	nb_{it} DHA	nb_{it} CCR-DHA	nb_{it} ZF-DHA
Unipolar	3	4	0.995	0.995	0.995	4.3	3.2	1.6
Unipolar	4	5	0.998	0.997	0.998	6.3	5.5	1.8
Unipolar	5	7	0.994	0.994	0.995	14.5	14.5	5.1
Unipolar	6	9	0.981	0.977	0.991	31.5	32	16.4
Unipolar	7	11	0.933	0.920	0.974	65	66	42.5
Bipolar	3	3	1.000	1.000	1.000	2.8	1.9	0.9
Bipolar	4	5	1.000	1.000	1.000	6.1	3.7	2.4
Bipolar	5	6	1.000	1.000	1.000	9.8	5.9	2.5
Bipolar	6	8	0.995	1.000	1.000	21.9	13.6	6.5
Bipolar	7	10	0.971	0.993	0.998	46.5	39.4	24.6
Gaussian	3	5	0.787	0.787	0.787	7.3	4.7	3.9
Gaussian	4	6	0.815	0.815	0.815	11.3	7.5	5.7
Gaussian	5	7	0.830	0.830	0.831	16.8	11.7	9.0
Gaussian	6	8	0.829	0.839	0.840	25.2	18.7	14.2
Gaussian	7	9	0.819	0.840	0.844	36.6	29.0	22.4

요약: ZF-DHA가 높은 성공확률과 적은 반복을 동시에 잡습니다.

6.3 IIMSA · CCR-IIMSA · ZF-IIMSA

원문 p.10 (Section V-D3), 원문 Fig. 7, p.9

같은 설정($L_{IIMSA} = L_{DHA} = 22.5\sqrt{2N}$, $\sigma^2 = 0.02$)에서 IIMSA 계열을 봅니다. 질문은 “DHA에서 얻은 결론이 IIMSA에도 적용되는가”입니다.

Fig. 7(a)-(c) — 성공확률. 세 IIMSA 변종이 거의 똑같습니다. Unipolar와 Bipolar는 모든 SF에서 $P_s \approx 1$ 에 붙어 있고, Gaussian만 SF에 따라 넓게 퍼집니다(가우시안 코드를 무작위로 뽑기 때문).

여기서 6.2절과 비교하면 **IIMSA의 진짜 가치**가 드러납니다. Unipolar $SF = 7$ 에서 DHA는 P_s 가 **0.933** 으로, CCR-DHA는 **0.920** 까지 떨어졌는데, **IIMSA 계열은 세 변종 모두 0.99** 근처를 유지합니다. Bipolar $SF = 7$ 도 DHA는 0.971인데 IIMSA는 1.000입니다. 즉 S 를 제대로 추정하는 것만으로, 가장 어려운 조건에서 무너지지 않습니다.

Fig. 7(d)-(f) — 반복 횟수. 여기서는 뚜렷한 차이가 납니다. ZF-IIMSA가 모든 SF에서 나머지 둘을 앞섭니다. 고전 ZF의 정확한 검출이 그로버의 반복 필요량을 줄여 주기 때문입니다. 그 다음이 CCR-IIMSA로, 검출이 덜 정확해도 여전히 원본 IIMSA보다 낮습니다. 다만 Unipolar에서는 CCR-IIMSA가 IIMSA와 비슷하고(또 그 CCR-Unipolar 문제), Bipolar·Gaussian에서는 확실히 앞섭니다.

표 3 — Fig. 7 읽은 값 ($\sigma^2 = 0.02$, 그래프에서 읽은 근삿값)

코드	SF	N	P_s (IIMSA 계열 공통)	nb_{it} IIMSA	nb_{it} CCR-IIMSA	nb_{it} ZF-IIMSA
Unipolar	3	4	0.995	4.8	3.5	2.0
Unipolar	4	5	0.998	6.5	5.5	1.8
Unipolar	5	7	0.996	13.0	12.8	5.2
Unipolar	6	9	0.992	27.5	26.5	13.8
Unipolar	7	11	0.994-0.997	56	56	35.5
Bipolar	3	3	1.000	3.0	2.8	1.1
Bipolar	4	5	1.000	6.5	4.2	2.8
Bipolar	5	6	1.000	9.4	6.5	2.8
Bipolar	6	8	1.000	19.2	12.7	6.0
Bipolar	7	10	1.000	40.3	34.5	20.6
Gaussian	3	5	0.786	6.9	5.3	4.4
Gaussian	4	6	0.817	10.1	7.9	6.1
Gaussian	5	7	0.830	14.4	11.6	8.8
Gaussian	6	8	0.842	21.1	17.5	13.2
Gaussian	7	9	0.847	30.1	26.0	19.5

요약: ZF-IIMSA도 높은 성공확률 + 적은 반복입니다. 성공확률은 다른 방법과 비슷하지만 nb_{it} 에서 확실히 앞서고, 이 우위는 SF가 높은 영역에서 더 중요해집니다 — 조밀한 IoT 네트워크에 유망하다는 뜻입니다.

6.4 복잡도를 숫자로 — 정말 고전보다 빠른가

이 절은 원문에 없습니다. 위 표들과 논문의 복잡도 식으로 제가 직접 계산해 본 것이니, 원문 인용 시에는 주의하세요.

복잡도 주장을 정리하면 이렇습니다.

방법	복잡도
고전 ML (완전탐색)	$O(2^N)$
그로버 (해 개수 S 알 때)	$L_{\text{opt}} = \lfloor \frac{\pi}{4} \sqrt{2^N/S} \rfloor$

방법	복잡도
BBHT	$\leq \frac{9}{2}\sqrt{K}$
DHA	$\leq 22.5\sqrt{K}$
IIMSA	예산 L_{IIMSA} (공정 비교 위해 $22.5\sqrt{2^N}$ 로 설정)

여기서 **함정 하나**. 가장 큰 경우인 Unipolar $SF = 7$, $N = 11$ 을 넣어 봅시다. $K = 2^{11} = 2048$ 입니다.

- 고전 ML: **2048회** 메트릭 계산
- DHA 이론 상계: $22.5\sqrt{2048} \approx 1018$ 회

상계만 보면 고전 대비 2배 남짓입니다. 점근적으로는 $O(2^N) \rightarrow O(2^{N/2})$ 인 이차 향상이 맞지만, 상수 22.5가 작은 N 에서는 이득을 거의 다 먹어 버립니다. 이걸 그로버 응용을 볼 때 항상 확인해야 하는 지점입니다.

그런데 실측값은 전혀 다릅니다.

- DHA 실측: 약 65회
- IIMSA 실측: 약 56회
- ZF-IIMSA 실측: 약 35.5회

상계는 최악의 경우이고, 실제로는 훨씬 일찍 끝납니다. 그리고 재미있는 사실 — 해가 하나($S = 1$)일 때의 이상적인 그로버 반복 수는 $\frac{\pi}{4}\sqrt{2048} \approx 35.5$ 입니다. **ZF-IIMSA의 실측 35.5는 이 이상적 값과 정확히 일치합니다.** 즉 ZF 초기화 + S 추정을 결합하면, 최솟값 탐색 전체가 “해를 하나 아는 상태에서 그로버 한 번 돌리기” 수준의 비용으로 끝난다는 뜻입니다. 이 논문의 결과 중 가장 인상적인 부분이라고 봅니다.

6.5 허용 반복 수 $L_{\max}$ 에 따른 IIMSA vs DHA

원문 p.10, p.12 (Section V-D4), 원문 Fig. 8, p.10

설정: $L \in \{1, \dots, L_{\max}\}$, $L_{\max} = 22.5\sqrt{2^N}$, 잡음 $\sigma = 0.1$, $SF = 6$. DHA 계열은 주황, IIMSA 계열은 파랑으로, Fig. 6·7과 색을 맞췄습니다.

기본 추세: L 이 커지면 P_s 도 커지고 nb_{it} 도 커집니다. 시도를 더 허용하니 해를 찾을 확률은 오르지만, 평균 반복 수도 늘어나는 당연한 교환입니다.

Fig. 8(a)-(c) — 성공확률: IIMSA가 DHA를 앞섭니다. 원본끼리도 그렇고, CCR/ZF로 개선한 버전끼리도 그렇습니다. IIMSA가 각 단계에서 해의 개수 추정을 활용해 더 효율적으로 탐색하기 때문입니다. ZF-IIMSA가 모든 알고리즘 중 최고이고, 이 추세는 모든 코드 타입에서 관찰됩니다.

Fig. 8(d)-(f) — 반복 횟수: 여기서 추세가 뒤집힙니다. L 이 작을 때는 고전 보조를 받은 DHA(ZF-DHA, CCR-DHA)가 IIMSA보다 nb_{it} 이 적습니다. 왜일까요?

저자들의 설명이 명쾌합니다. IIMSA는 처음부터 L_{opt} 만큼 큰 반복 수로 시작합니다. 반면 DHA는 $L \in \{0, \dots, \lfloor m \rfloor\}$ 에서 뽑는데 m 이 1부터 서서히 커집니다. while 루프는 반복 수가 한도를 넘으면 멈추므로,

DHA는 통계적으로 IIMSA보다 작은 값에서 한도를 넘겨 버립니다. 즉 DHA의 nb_{it} 이 작아 보이는 건 잘해서가 아니라 일찍 포기해서입니다 — 그래서 같은 그림의 P_s 는 낮습니다.

이 현상은 L 이 커지면 사라집니다. L 이 크면 while 루프가 “한도 초과“가 아니라 “최솟값 발견“으로 종료될 가능성이 높아지므로, 위의 소규모 효과가 더 이상 지배적이지 않습니다. 이 패턴은 모든 혼합 고전 알고리즘에서 동일합니다.

코드 타입별 최종 수치 (높은 $L_{\max}$ 영역):

코드	최대 P_s	최대 nb_{it}
Bipolar	0.88	18
Unipolar	0.7	29
Gaussian	< 0.5	25

Bipolar가 최고인 이유는 역시 $\mathbf{C} \in \{-1, 1\}$ 의 큰 유클리드 거리 → 정확한 검출 → 잡음 내성입니다. 그리고 nb_{it} 도 가장 작는데, 논리가 재미있습니다 — Bipolar는 P_s 가 높아서 $L_{\max}$ 에 도달하는 경우 자체가 드물고, 그래서 반복 수 기댓값이 줄어듭니다.

고전 수신기의 기여: CCR은 ZF와 경쟁하기엔 부족하지만, 원본 알고리즘보다는 P_s 가 높습니다. 즉 CCR도 탐색 초기화에 긍정적으로 기여해 정확한 검출에 필요한 nb_{it} 을 줄입니다. 결과적으로 CCR도 원본 DHA·IIMSA를 능가합니다.

$L > 20$ 구간에서는 ZF·IIMSA가 비슷한 성능을 더 작은 nb_{it} 로 달성하고, ZF-DHA가 근소한 차이로 뒤따릅니다. 저자들의 결론: ZF는 QMSA에 정확한 초기화 집합을 제공하는 훌륭한 방법입니다.

6.6 잡음 있는 양자 하드웨어

원문 p.12 (Section V-D5), 원문 Fig. 9, p.11

지금까지는 완벽한 양자 하드웨어를 가정했습니다. 이제 현실을 넣습니다.

잡음 모델: TDCh(total depolarizing channel, 전체 탈분극 채널) [21]. 현재 상태 ρ 가 확률 λ 로 최대혼합상태(maximally mixed state, 모든 기저 상태의 균등중첩에 해당)로 사상되고, 확률 $1 - \lambda$ 로 그대로 유지됩니다. 직관적으로 “확률 λ 로 계산이 통째로 리셋되어 아무 정보 없는 상태가 된다“는 모델입니다.

설정: $\sigma = 0.1$, $SF = 6$, $L \in \{1, \dots, L_{\max}\}$, $L_{\max} = 22.5\sqrt{2^N}$. TDCh 계수 $\lambda \in \{0, 0.05, 0.1\}$ 세 가지를 IIMSA와 DHA 각각에 적용. $\lambda = 0$ 이 앞 절의 완벽한 경우로, 기준선 역할을 합니다.

결과 1 — 예상대로, 잡음 있는 하드웨어는 모든 코드 타입에서 성능을 떨어뜨립니다. 하지만 DHA와 IIMSA가 다르게 반응합니다.

- DHA (Fig. 9(a)-(f)): 두 영역으로 나뉩니다. L 이 낮을 때는 하드웨어 잡음의 영향이 무시할 만하거나 아예 없고, L 이 높을 때만 영향을 받습니다.
- IIMSA (Fig. 9(g)-(l)): L 의 전 구간에서 영향이 나타납니다.

왜 이런 차이가? 저자들의 설명은 이 논문 전체에서 가장 통찰력 있는 대목입니다. 요지는 DHA가 S 를 눈감고 찍는 탓에 이미 자기 실패율이 크다는 것입니다. S 를 잘못 찍으면 그로버가 유효한 해를 못 내놓는데, 이 일이

위낙 자주 일어나서 TDCh 잡음으로 해를 놓치는 것쯤은 그 안에 묻혀 버립니다. 시도 횟수가 적은 낮은 L 구간에서 특히 그렇습니다. 반대로 IIMSA는 적절한 S 를 쓰므로 매 반복이 효율적이고, 그래서 TDCh 오류가 전 구간에서 고스란히 드러납니다.

한 줄로 하면: **잘 만든 알고리즘일수록 하드웨어 잡음에 더 민감해 보입니다.** 역설 같지만 당연합니다 — 못 만든 알고리즘은 이미 자기 노이즈에 잠겨 있어서 하드웨어가 좀 더 흔들어도 티가 안 날 뿐입니다. 벤치마크를 읽을 때 조심해야 할 함정이기도 합니다.

결과 2 — 잡음 있는 하드웨어를 고려해도 IIMSA는 여전히 DHA를 능가합니다. CCR/ZF 초기화도 여전히 유효한 전략입니다. 잡음 오류가 있다고 해서 “정확한 초기화와 파라미터 설정이 탐색을 효율적으로 겨냥해 준다”는 사실이 바뀌지는 않기 때문입니다. **결론: 잡음 있는 하드웨어에 배치해도 ZF-IIMSA는 매우 유망합니다.**

7. 결론

원문 p.12-13 (Section VI)

저자들의 마무리를 정리하면:

1. **AUD 문제에 그로버 알고리즘의 적응과 확장을 제안했습니다.** DHA 같은 기존 알고리즘도 이 문제를 다룰 수 있지만 복잡도가 여전히 무시할 수 없는 수준이었습니다.
2. **IIMSA — 시스템의 통계적 거동을 활용해 해의 개수를 평가하는 새 알고리즘. 특히 SF가 높은 영역에서 DHA 대비 우수합니다.**
3. **CCR-ZF 고전 방법으로 양자 탐색 기준을 정확히 초기화.** 이를 통해 그로버 반복 수를 더 작은 L_{IIMSA} 에서 시작할 수 있었고, 성능 저하 없이(오히려 약간 개선하면서) 반복을 줄였습니다.
4. **이 가벼운 고전 계산이 주는 이득은 SF가 높을수록 커집니다 → 조밀한 IoT 네트워크에 유망합니다.**
5. 완벽한 하드웨어에서 얻은 이 결과가 잡음 있는 양자 하드웨어 배치에서도 유지됨을 보였습니다.

향후 과제: 이 알고리즘 위에 오류 완화(error mitigation) 전략을 엮는 것이 다음 단계라고 밝힙니다.

8. 이 프로젝트에 어떻게 쓰이나

이 논문은 우리 저장소에 **직접적으로** 맞물립니다. `project1/hardware/grover_min_ip.v` 의 헤더 주석이 이미 이렇게 시작합니다:

`grover_min_ip.v` — Grover 기반 "최솟값 탐색" IP (Dürr - Høyer, DHA)

즉 이 논문이 개선하려는 바로 그 알고리즘을 우리가 이미 구현해 두었습니다. 이 논문은 그 IP의 다음 로드맵을 통째로 제공합니다.

8.1 이미 맞아 있는 것

`grover_min_ip.v` 를 논문 용어로 다시 읽으면 대응이 정확합니다.

논문	grover_min_ip.v
임계값 $\hat{\delta} / U[i_x]$	reg signed [VW-1:0] T
비교기 오라클 ($U[x] < U[i_x]$ 마킹)	S_ORACLE: if (value[i] < T) amp[i] <= -amp[i];
확산 연산자 D (평균 반사)	S_SUM + S_DIFF: amp[i] <= two_mean - amp[i];
측정	S_MEAS: 진폭 argmax + LFSR 동점 처리
고전 검증 + 기준 갱신 (Algorithm 2 Step 10)	S_UPD: if (value[cand] < T) begin T <= value[cand]; best
외부 루프	round / ROUNDS
회당 그로버 반복 j	R_GROVER (고정 2)
균등중첩 재준비 (Algorithm 2 Step 5)	S_REINIT

주석에 적힌 “★ 이 파일의 핵심은 S_ORACLE 상태의 ‘비교기 오라클’ 한 줄이다 ★” 는 정확히 논문 5.2.2절(원문 Eq. 12, Fig. 3)이 말하는 것과 같습니다. 이 논문을 읽는 가장 큰 소득은 **그 한 줄이 왜 그렇게 중요한지, 그리고 진짜 양자 회로에서는 그 한 줄이 무엇으로 전개되는지를** 알게 되는 것입니다.

8.2 지금 코드에 있는 문제 — 논문이 정확히 지적하는 것

T_INIT = 16'sh7FFF 로 시작하는 첫 라운드를 논문의 눈으로 봅시다.

모든 value[i] 가 0x7FFF 보다 작으므로 오라클이 **전체 진폭의 부호를 반전**시킵니다. 논문 용어로 $S = K$, 즉 해의 개수가 데이터베이스 크기와 같습니다. 이건 4.3절의 경고 — “ S 가 데이터베이스 크기의 절반보다 크면 원치 않는 경우가 증폭되어 탐색이 엉뚱한 방향으로 간다” — 의 가장 극단적인 경우입니다.

실제로 어떻게 되는지 손으로 따라가 보면: 모든 $\text{amp}[i] = -\text{INIT_AMP} \rightarrow \text{mean} = -\text{INIT_AMP} \rightarrow \text{two_mean} = -2 \cdot \text{INIT_AMP} \rightarrow \text{amp}[i] \leftarrow \text{two_mean} - \text{amp}[i] = -\text{INIT_AMP}$. **아무것도 안 변합니다.** 전역 위상만 바뀔 뿐이라 확산이 할 일이 없습니다. R_GROVER = 2 번을 돌려도 마찬가지입니다.

결과적으로 첫 라운드의 측정은 **전 인덱스가 동점인** 상태에서 이루어지고, R_GROVER = 2 회의 그로버 반복이 **완전히 낭비**됩니다. 재미있는 건, 이 낭비된 라운드가 결과적으로 하는 일이 “**무작위로 인덱스 하나 뽑기**” 라서 **DHA Algorithm 2의 Step 3(무작위 초기화)**과 **우연히 같아진**다는 점입니다. 그래서 코드는 “동작합니다 — 다만 그로버를 헛돌린 대가로요.

논문이 제시하는 해법이 정확히 **4.4절의 CCR/ZF 초기화**입니다. T 를 0x7FFF 가 아니라 **고전 전처리가 찾아준 δ_{ZF}** 에서 출발시키면, 첫 라운드부터 S 가 작아져 그로버가 실제로 일을 합니다.

부수적으로, 현재 동점 처리 로직(`else if (cur_abs == max_abs && lfsr[0]) cand = i;`)은 루프 안에서 lfsr[0] 이 안 바뀌므로 전 인덱스 동점 시 **인덱스 0 또는 $N - 1$ 만** 뽑습니다. DHA의 균등 초기화를 제대로 흉내 내려면 이 부분도 손봐야 합니다. (지금 당장 고치라는 게 아니라, 논문대로 갈 때 걸릴 지점이라 적어 둡니다.)

8.3 논문을 따라가려면 하드웨어에 무엇이 필요한가

우선순위 순으로 정리합니다.

① 임계값 레지스터를 포트로 빼기 — 가장 싸고 효과 큰 개선

지금 T_INIT 은 파라미터입니다. 이걸 입력 포트 t_init 으로 바꾸면 CCR/ZF 하이브리드를 실험할 수 있습니다. 처음에는 테스트벤치(grover_tb.v)가 δ_{ZF} 를 계산해 넣어 주는 것으로 충분합니다. RTL 한 줄 수준의 변경으로 논문의 두 번째 기여를 재현할 수 있습니다.

② nb_it 카운터 추가 — 논문의 핵심 지표

논문의 모든 그래프의 y 축이 nb_{it} 입니다. 지금 IP는 이 값을 세지 않습니다. S_DIFF 에서 증가하는 카운터를 하나 만들고 출력 포트로 노출하면, 우리 에뮬레이터로 Fig. 6·7 같은 곡선을 직접 뽑을 수 있습니다. P_s 는 테스트벤치에서 시드를 바꿔 여러 번 돌려 $\hat{b} = b$ 비율을 세면 됩니다.

③ R_GROVER 고정값을 알고리즘에 맞게 바꾸기 — DHA와 IIMSA가 갈리는 지점

- DHA를 제대로 하려면: m 레지스터(초기 1), $\lambda = 6/5$, $j \sim \text{Uniform}\{0, \dots, \lfloor m \rfloor\}$, 실패 시 $m \leq \min(\lambda \cdot m, \text{sqrt}(K))$. LFSR은 이미 있으니 j 뽑기는 쉽습니다. $\lambda = 6/5 = 1.2$ 곱은 고정소수점 곱 한 번($m \cdot 6/5$ 또는 Q 포맷 상수 곱)이면 됩니다.
- IIMSA를 하려면: $L_{\text{opt}} = \lfloor \frac{\pi}{4} \sqrt{2^N/S} \rfloor$ 가 필요합니다. 제공근과 나눗셈이 들어가는데, N 이 작으면 그냥 LUT가 정답입니다. $S \in \{1, \dots, K\}$ 에 대해 $L_{\text{opt}}[S]$ 를 미리 계산해 ROM에 넣으면 사이클 1회에 끝납니다. $K = 8$ 이면 엔트리 8개짜리 ROM입니다.
- 그리고 S 추정기가 필요합니다. 논문은 “오프라인 통계 분석”이라고만 하지 구체적 방법을 주지 않습니다. 우리 에뮬레이터에서는 두 가지 길이 있습니다. (a) **정직한 길**: 잡음 분산·코드북으로부터 S 의 기댓값 LUT를 미리 만들어 넣는다. (b) **연구용 상한 확인**: 시뮬레이션이니 그냥 세어 버린다 — 물론 이건 실제 양자 시스템에서 불가능하므로 IIMSA의 성능 상한을 재는 용도로만 유효합니다. 이 구분을 흐리면 결과가 무의미해지니 문서에 반드시 명시하세요.

④ value[] 배열을 메트릭 데이터패스로 교체 — 진짜 AUD를 하려면

이게 가장 큰 작업이고, 개념적으로 가장 중요합니다. 지금 IP는 $\text{value}[0..7] = \{5, 2, 8, 1, 7, 3, 6, 4\}$ 를 배열로 들고 있습니다. 하지만 논문의 오라클에는 저장된 배열이 없습니다(5.2.4절). $U[x]$ 는 인덱스 x (= 활성 벡터 $\mathbf{b}$)로부터 측석에서 계산되는 값입니다.

AUD로 바꾸려면 $\text{value}[i]$ 를 다음 데이터패스로 대체합니다:

1. 인덱스 i 의 비트가 곧 활성 벡터 $\mathbf{b} \rightarrow$ 활성 사용자들의 코드워드를 칩별로 합산 ($\mathbf{b} \cdot \mathbf{C}$). Unipolar $N = 4$, $SF = 3$ 이면 원문 Eq. 14 그대로, 덧셈 몇 번입니다.
2. $\mathbf{y} = \mathbf{b} \cdot \mathbf{C}$ (논문의 A_f) — 칩별 뺄셈 SF 개.
3. $\|\cdot\|_2^2$ (논문의 N_f) — 제공 SF 개 + 합.
4. $< T$ 비교 (논문의 U_δ) $\rightarrow$ 마킹 (논문의 M_f).

$SF = 3$ 이면 이건 곱셈기 3개와 덧셈기 몇 개입니다. FPGA에서 전혀 부담이 아닙니다. 그리고 코드북 $\mathbf{C}$ 와 수신 신호 $\mathbf{y}$ 는 원문 Eq. 13의 양자화 규칙($p = 2$)으로 고정소수점화하면 되는데, 이미 우리 IP가 Q4.16을 쓰고 있으니 포맷 변환 문제일 뿐입니다.

⑤ 언컴퓨테이션은 필요 없습니다 — 중요한 구분

원문 Fig. 4는 $A_f \rightarrow N_f \rightarrow U_\delta \rightarrow M_f \rightarrow U_\delta \rightarrow N_f \rightarrow A_f$ 로 되돌리는(uncompute) 절반을 갖습니다. 우리 IP는 상태벡터 시뮬레이터이지 게이트 단위 양자 회로 에뮬레이터가 아닙니다. 진폭 배열을 직접 들고 있으므로 오라클의 술어(predicate)만 계산하면 되고, ancilla도 언컴퓨트도 필요 없습니다.

이 구분을 명확히 해 두는 게 좋습니다. 만약 프로젝트 목표가 **게이트 단위 회로 에뮬레이션**으로 올라간다면 그때는 원문 Fig. 3이 청사진이 됩니다 — Reference/Index/Ancilla/Mark 4개 레지스터, 자리빌림 체인, $|-\rangle$ 로의 CNOT, 그리고 미러 이미지 언컴퓨터. 두 목표는 **회로 구조가 완전히 다르니, 어느 쪽을 만드는지 먼저 정하고 들어가세요.**

⑥ 종료 조건: 고정 $\text{ROUNDS} \rightarrow \hat{S} = 0$ 조기 종료 + 예산

논문의 IIMSA는 $\hat{S} = 0$ 이면 즉시 끝냅니다(더 작은 값이 없으니 현재 $\hat{\delta}$ 가 최소). 우리 IP는 $\text{ROUNDS} = 16$ 을 무조건 다 돕니다. S 추정기(③)가 생기면 조기 종료를 붙일 수 있고, 예산은 논문대로 $22.5\sqrt{2^N}$ 로 두면 됩니다.

8.4 검증 계획 제안

논문이 우리에게 주는 가장 실용적인 선물은 **재현 가능한 목표 수치**입니다. 예를 들어 이런 단계로 진행할 수 있습니다.

1. nb_it 카운터 추가 → **검증**: 현재 value[] 데모(최솟값 1 @ index 3)에서 done 시점의 nb_it 이 $\text{ROUNDS} \times \text{R_GROVER} = 32$ 인지 확인.
2. T_INIT 을 포트화하고, 테스트벤치에서 $T = 6$ (= value[6] 근처의 좋은 초기값)으로 시작 → **검증**: S 가 줄어들어 nb_it 이 감소하고 result = 3 이 유지되는지 확인. 이것이 CCR/ZF 하이브리드의 축소판 실험입니다.
3. R_GROVER 를 L_{opt} LUT로 교체 → **검증**: 같은 P_s 를 더 적은 nb_it 으로 달성 (논문 6.1절의 IIMSA vs DHA 결론 재현).
4. value[] 를 AUD 메트릭 데이터패스로 교체 (Unipolar, $SF = 3, N = 4$, 원문 Eq. 14) → **검증**: σ^2 를 훑으며 P_s 곡선을 뽑아 원문 Fig. 5(a)의 추세(분산 증가 → P_s 감소)와 비교. $SF = 3, N = 4$ 는 원문 Table I의 첫 행이라 조건이 정확히 맞습니다.

34단계까지 가면 **원문 Fig. 57을 우리 FPGA에서 재현**한 셈이 되고, 그게 이 인턴십 프로젝트의 자연스러운 결승선으로 보입니다. 논문 저자들이 32큐비트를 넘겨 MATLAB으로 시뮬레이션했다는 점(5.1절)을 생각하면, **하드웨어 가속으로 더 큰 N 을 돌리는 것은 그 자체로 할 말이 있는 결과**입니다.

8.5 이 논문에서 반드시 가져갈 개념 세 가지

1. **비교기가 오라클이 된다.** 정답을 몰라도 “임계값보다 작은가”만 판정할 수 있으면 그로버를 쓸 수 있습니다. 이것이 그로버를 교과서 예제에서 실제 문제로 옮기는 유일한 다리입니다. (원문 Eq. 12, Fig. 3)
2. **비자명한 조건을 자명한 조건의 반복으로 바꾼다.** “최솟값인가?”는 못 물어보지만 “ δ 보다 작은가?”는 물어볼 수 있습니다. δ 를 계속 낮추면 최솟값에 닿습니다. 이것이 DHA/IIMSA의 전부입니다. (원문 Section IV-A)
3. **그로버는 “ S 를 안다”에 목숨을 건다.** $L_{\text{opt}} = \lfloor \frac{\pi}{4} \sqrt{K/S} \rfloor$ 는 S 를 알아야 계산됩니다. 모르면 무작위로 찍어야 하고(DHA), 그게 대부분의 낭비를 만듭니다. S 를 추정하는 것(IIMSA)과 좋은 δ 에서 출발하는 것(ZF)이 이 논문 성능 향상의 전부이고, 둘 다 “양자를 더 잘하기”가 아니라 “고전 지식을 양자에 먹이기”입니다. 하이브리드 양자-고전 알고리즘 설계의 교과서적 사례입니다.

부록: 기호 요약

기호	의미
N	사용자(단말) 수 = 큐비트 수
SF	확산 계수 (코드워드당 칩 수)
$K = 2^N$	검색 공간 크기 (= 가능한 활성 조합 수)
S	해의 개수 (현재 임계값보다 거리가 작은 후보 수)
$\mathbf{b} \in \{0, 1\}^N$	활성 벡터 (참값)
$\hat{\mathbf{b}}$	추정 활성 벡터
$\mathbf{C} \in \mathbb{R}^{N \times SF}$	코드북
$\mathbf{C}_C$	의사역행렬 $(\mathbf{C}^T \mathbf{C})^{-1} \mathbf{C}^T$
$\mathbf{y} \in \mathbb{R}^{SF}$	수신 신호
$\mathbf{y}_q$	양자화된 수신 신호
$\mathbf{w}$	AWGN 잡음, $\mathcal{N}(0, \sigma^2)$
σ^2	잡음 분산
T	CCR의 판정 임계값
$\delta, \hat{\delta}, \delta_x$	최솟값 탐색의 기준 거리 (임계값)
$\delta_{CCR}, \delta_{ZF}$	고전 전처리가 준 초기 임계값
$f(x)$	$\ \mathbf{y} - \mathbf{b} \cdot \mathbf{C}\ _2^2$ (후보 $x = \mathbf{b}$ 의 거리)
$\mathcal{O}, \mathcal{D}, \mathcal{G}$	오라클, 확산 연산자, 그로버 연산자
L_{opt}	최적 그로버 반복 수 $\lfloor \frac{\pi}{4} \sqrt{K/S} \rfloor$
$L_{\text{BBHT}}, L_{\text{DHA}}, L_{\text{IMSA}}$	각 알고리즘의 반복 예산
L_{max}	허용 반복 상한 $22.5\sqrt{2^N}$
nb_{it}	그로버 반복 총 횟수 (복잡도 지표)
P_s	성공확률 ($\hat{\mathbf{b}} = \mathbf{b}$ 인 비율)
m, λ	DHA/BBHT의 반복 범위와 확대 인자 ($\lambda = 6/5$ 또는 $4/3$)
I, p, Z	정수부 비트 수, 소수부 정밀도(= 2), 레지스터 크기 $Z = I + p$
λ (Section V-D5)	TDCh 탈분극 확률 — 위의 λ 와 다른 기호이니 주의

작성 기준: 원문 PDF 13쪽 전체. 수식 번호는 원문 표기를 따랐습니다. 표 2·표 3의 수치는 원문 그래프에서 읽은 근삿값이므로 인용 시 원문 그림을 확인하세요.